The U-Net Architecture for Wildfire Segmentation

Introduction

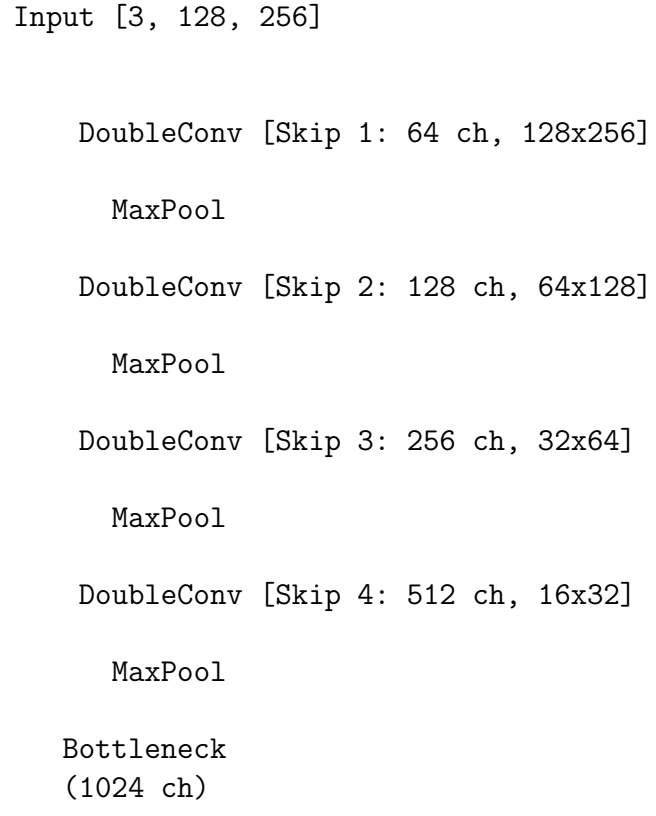
The data pipeline is verified and locked. Your batches have the exact tensor dimensions required: inputs of shape $[8, 3, 128, 256]$ and target masks of shape $[8, 1, 128, 256]$, with normalized physical temperatures and clean binary labels.

The next step is constructing the U-Net architecture in PyTorch.

1. The Architectural Fundamentals: Why U-Net?

Standard convolutional classification networks (such as ResNet or VGG) downsample an image repeatedly and pass the result through a Global Average Pooling layer to output a single label: "Does this image contain a fire? Yes or No."

That approach fails for disaster management because it destroys spatial coordinates. We need **semantic segmentation**: classifying every individual pixel on a 128×256 grid as either active combustion (1.0) or background terrain (0.0).



```
   ConvUp

Concat [512 + 512] > DoubleConv (512)

   ConvUp

Concat [256 + 256] > DoubleConv (256)

   ConvUp

Concat [128 + 128] > DoubleConv (128)

   ConvUp

Concat [64 + 64] > DoubleConv (64)

  Conv 1x1


Output Logits [1, 128, 256]
```

The Contracting Path (Encoder)

The left side extracts radiometric features across expanding receptive fields:

- **Double Convolution:** Two consecutive 3×3 convolutional layers, each followed by Batch Normalization and a Rectified Linear Unit (ReLU):

  $$\text{Conv3x3} \rightarrow \text{BatchNorm2d} \rightarrow \text{ReLU} \rightarrow \text{Conv3x3} \rightarrow \text{BatchNorm2d} \rightarrow \text{ReLU}$$

  - The first convolution detects localized edges and steep thermal contrasts (ΔT).
  - The second convolution refines those spatial patterns.
  - Batch Normalization prevents internal covariate shift by ensuring activations maintain zero mean and unit variance throughout the forward pass.

- **Downsampling (2×2 Max Pooling):** Reduces the spatial dimensions (H, W) by half while doubling the channel depth. At the deepest point (Level 4), the spatial resolution is 8×16 with 1024 channels.

The Bottleneck

The lowest layer processes high-level contextual information. A single pixel at the bottleneck represents a large geographic area (16×16 original pixels), enabling the model to determine whether a heat anomaly is an inland desert plateau, coastal farmland, or a mountain forest ridge.

The Dilemma of Downsampling for Fires

Downsampling reduces spatial precision. A small wildfire might occupy only a 1×1 pixel anomaly in the original satellite scan. By the time that tensor passes through four 2×2 pooling operations ($16\times$ spatial reduction), a single-pixel fire is diluted and lost.

The Expanding Path and Skip Connections (The Solution)

The right side restores the original spatial resolution:

- **Transposed Convolution (ConvTranspose2d):** Doubles the height and width while halving the channel depth (e.g., from 8×16 with 1024 channels to 16×32 with 512 channels).
- **Skip Connections (`torch.cat`):** Before convolving the upsampled feature map, the network copies the high-resolution feature map directly from the corresponding encoder stage and concatenates them along the channel dimension:

  Decoder Channels = Upsampled Channels (512)+Encoder Skip Channels (512) = 1024 channels

- The decoder gets both the high-level context (from the deep layers) and the exact coordinate boundaries (from the shallow layers).
- **Final Projection (1×1 Convolution):** Maps the final 64 feature channels to a single output channel ($C_{\text{out}} = 1$), producing unconstrained raw classification scores (logits) for every pixel.

2. The PyTorch Implementation (`unet_model.py`)

Create a new file named `unet_model.py` in your project folder:

```
import torch
import torch.nn as nn


class DoubleConv(nn.Module):
  """A sequence of two 3x3 convolutions, each followed by
     BatchNorm and ReLU.

  Formula: [Conv2d(3x3) -> BatchNorm2d -> ReLU] x 2
  Padding=1 preserves spatial dimensions (H, W).
  """

  def __init__(self, in_channels, out_channels):
    super(DoubleConv, self).__init__()
    self.conv_block = nn.Sequential(
        nn.Conv2d(
            in_channels,
            out_channels,
            kernel_size=3,
            padding=1,
```

```
            bias=False,  # Bias is redundant when followed
                immediately by BatchNorm
        ),
        nn.BatchNorm2d(out_channels),
        nn.ReLU(inplace=True),
        nn.Conv2d(
            out_channels, out_channels, kernel_size=3, padding=1,
                bias=False
        ),
        nn.BatchNorm2d(out_channels),
        nn.ReLU(inplace=True),
    )

  def forward(self, x):
    return self.conv_block(x)


class UNet(nn.Module):
  """Standard U-Net architecture adapted for multi-spectral
     thermal segmentation.

  Inputs:  Tensor of shape [Batch, in_channels, 128, 256]
  Outputs: Tensor of shape [Batch, out_channels, 128, 256] (Raw
     Logits)
  """

  def __init__(self, in_channels=3, out_channels=1, base_features
     =64):
    super(UNet, self).__init__()

    f = base_features  # 64 base filters

    # 1. Contracting Path (Encoder)
    self.inc = DoubleConv(in_channels, f)  # 3 -> 64
    self.down1 = nn.Sequential(
        nn.MaxPool2d(kernel_size=2, stride=2), DoubleConv(f, f *
           2)  # 64 -> 128
    )
    self.down2 = nn.Sequential(
        nn.MaxPool2d(kernel_size=2, stride=2),
        DoubleConv(f * 2, f * 4),  # 128 -> 256
    )
    self.down3 = nn.Sequential(
        nn.MaxPool2d(kernel_size=2, stride=2),
        DoubleConv(f * 4, f * 8),  # 256 -> 512
    )

    # 2. Bottleneck
    self.down4 = nn.Sequential(
        nn.MaxPool2d(kernel_size=2, stride=2),
        DoubleConv(f * 8, f * 16),  # 512 -> 1024
```

```
        )

        # 3. Expanding Path (Decoder)
        # Level 4 Up: 1024 -> 512
        self.up1 = nn.ConvTranspose2d(f * 16, f * 8, kernel_size=2,
            stride=2)
        self.conv1 = DoubleConv(
            f * 16, f * 8
        )  # Concatenated: 512 (up) + 512 (skip) = 1024 -> 512

        # Level 3 Up: 512 -> 256
        self.up2 = nn.ConvTranspose2d(f * 8, f * 4, kernel_size=2,
            stride=2)
        self.conv2 = DoubleConv(
            f * 8, f * 4
        )  # Concatenated: 256 (up) + 256 (skip) = 512 -> 256

        # Level 2 Up: 256 -> 128
        self.up3 = nn.ConvTranspose2d(f * 4, f * 2, kernel_size=2,
            stride=2)
        self.conv3 = DoubleConv(
            f * 4, f * 2
        )  # Concatenated: 128 (up) + 128 (skip) = 256 -> 128

        # Level 1 Up: 128 -> 64
        self.up4 = nn.ConvTranspose2d(f * 2, f, kernel_size=2, stride
            =2)
        self.conv4 = DoubleConv(
            f * 2, f
        )  # Concatenated: 64 (up) + 64 (skip) = 128 -> 64

        # 4. Final Output Projection (1x1 Convolution)
        # Maps 64 feature maps to 1 channel of raw logits (No Sigmoid
            here; applied in Loss)
        self.out_conv = nn.Conv2d(f, out_channels, kernel_size=1)

    def forward(self, x):
        # Encoder with skip connections saved
        x1 = self.inc(x)  # Level 1: [B, 64, 128, 256]
        x2 = self.down1(x1)  # Level 2: [B, 128, 64, 128]
        x3 = self.down2(x2)  # Level 3: [B, 256, 32, 64]
        x4 = self.down3(x3)  # Level 4: [B, 512, 16, 32]

        # Bottleneck
        x5 = self.down4(x4)  # Bottleneck: [B, 1024, 8, 16]

        # Decoder with concatenation along channel dimension (dim=1)
        # Up 1
        d4 = self.up1(x5)  # [B, 512, 16, 32]
        d4 = torch.cat([x4, d4], dim=1)  # [B, 1024, 16, 32]
        d4 = self.conv1(d4)  # [B, 512, 16, 32]
```

```
        # Up 2
        d3 = self.up2(d4)  # [B, 256, 32, 64]
        d3 = torch.cat([x3, d3], dim=1)  # [B, 512, 32, 64]
        d3 = self.conv2(d3)  # [B, 256, 32, 64]

        # Up 3
        d2 = self.up3(d3)  # [B, 128, 64, 128]
        d2 = torch.cat([x2, d2], dim=1)  # [B, 256, 64, 128]
        d2 = self.conv3(d2)  # [B, 128, 64, 128]

        # Up 4
        d1 = self.up4(d2)  # [B, 64, 128, 256]
        d1 = torch.cat([x1, d1], dim=1)  # [B, 128, 128, 256]
        d1 = self.conv4(d1)  # [B, 64, 128, 256]

        # Final 1x1 Convolution
        logits = self.out_conv(d1)  # [B, 1, 128, 256]

        return logits


#
    ================================================================

# VERIFICATION & HARDWARE SANITY CHECK
#
    ================================================================

if __name__ == "__main__":
    # 1. Determine execution device
    device = torch.device("cuda" if torch.cuda.is_available() else
        "cpu")
    print(f"Executing architecture check on device: {device}")
    if device.type == "cuda":
        print(f"GPU Model: {torch.cuda.get_device_name(0)}")
        print(
            f"Available VRAM: {torch.cuda.get_device_properties(0).
                total_memory / (1024**3):.2f} GB"
        )

    # 2. Instantiate Model
    model = UNet(in_channels=3, out_channels=1, base_features=64).
        to(device)

    # 3. Calculate trainable parameter count
    total_params = sum(p.numel() for p in model.parameters() if p.
        requires_grad)
    print(
        f"Total Trainable Parameters: {total_params:,} (~{
            total_params * 4 / (1024**2):.2f} MB float32)"
```

```
)

# 4. Generate dummy mini-batch matching DataLoader format: [
    Batch=8, Channels=3, Height=128, Width=256]
dummy_input = torch.randn(8, 3, 128, 256, device=device)
print(f"\nForward pass with dummy tensor: {dummy_input.shape}")

# 5. Run Forward Pass
with torch.no_grad():
  output = model(dummy_input)

print(f"Output Tensor Shape [B, C, H, W]: {output.shape}")
assert output.shape == (
    8,
    1,
    128,
    256,
), "Output dimensions do not match expected shape!"

if device.type == "cuda":
  vram_used = torch.cuda.memory_allocated(0) / (1024**2)
  print(f"GPU VRAM Allocated during forward pass: {vram_used:.2
     f} MB")

print(
    "\nModel successfully built and verified! Ready for loss
       function integration."
)
```

3. Run the Verification

Execute the architecture test directly from your terminal:

```
python unet_model.py
```

Run this script to verify that the forward pass completes, the parameter count matches expectations, and the output tensor dimensions align with your ground truth masks.